

פרויקט הגמר — צ'אט מעל מסמכי החוג

עבודת צוות · 2-3 ימי עבודה · הגשה: ריפו ציבורי ב-GitHub

מה בונים

ממשק צ'אט בעברית שעונה על שאלות אמיתיות של סטודנטים מתוך שני מסמכים רשמיים של המכללה — השנתון של החוג למדעי המחשב, ותקנון הלימודים לתואר ראשון.

לא chatbot כללי. מערכת שעונה רק ממה שכתוב במסמכים, מציינת מאיפה לקחה כל תשובה, ואומרת בפירוש "לא מופיע במסמכים" כשהתשובה לא שם.

המשפט האחרון הוא החלק הכי חשוב בפרויקט. מערכת שממציאה תשובה על תקנון גרועה בהרבה ממערכת שלא עונה בכלל.

מה אתם מקבלים

בתיקייה `final-project/pdfs/`:

קובץ	עמודים
שנתון תשפז - מדעי המחשב.pdf	45
תקנון לתואר ראשון - תשפז.pdf	13

בסך הכל כ-58 עמודים, כ-233 אלף תווים של טקסט אמיתי (לא סריקות — אין צורך ב-OCR).

שימו לב לפני שאתם מתחילים: חילוץ נאיבי של הטקסט לא יעבוד. תגלו את זה בעצמכם בשלב 1 — פשוט אל תבנו שכבה שלמה מעל פלט שלא הסתכלתם עליו. תפתחו את הטקסט שחילצתם ותקראו אותו בעיניים לפני שממשיכים.

הסטאק המומלץ

זה המסלול שאנחנו ממליצים עליו — הוא נבדק, הוא עובד, והוא חוסך לכם החלטות שלא כדאי לשרוף עליהן זמן בפרויקט של שלושה ימים. אתם לא חייבים אותו. אם בחרתם אחרת, תכתבו משפט ב-README שמסביר למה.

אם אתם מתלבטים — קחו את ברירת המחדל ותמשיכו הלאה. בחירת ספריות היא לא החלק המעניין בפרויקט הזה.

שכבה	ברירת מחדל	הערה
שפה	Python 3.11 ומעלה	
חילוץ PDF	pdfplumber	Windows ו-Mac, עובד זהה
טיפול ב-RTL	python-bidi	
אינדקס חיפוש	SQLite FTS5	דירוג (bm25), מובנה, טוקניזר (unicode61)
מודל	anthropic, SDK, claude-sonnet-5	
ממשק	Streamlit	הרכיבים st.chat_input ו-st.chat_message
הרצה	מקומית בלבד	לא מעלים לאוויר — ראו סעיף מפתחות

מה שכדאי מאוד להימנע ממנו: embeddings, vector database, Postgres, pgvector, Chroma, torch. לא בגלל שהם כלים גרועים — בגלל שבקורפוס בגודל הזה הם תשתית מיותרת, ואחזור סמנטי בעברית הוא קרב שיאכל לכם יום שלם מתוך שלושה. אם אתם בכל זאת הולכים לשם, לכו בעיניים פקוחות ותסבירו את השיקול ב-README.

האינדקס שלכם הוא קובץ אחד בגודל כמה מאות קילובייט. זה בסדר גמור. חלק מהלמידה בפרויקט הזה היא לזהות מתי תשתית היא התשובה הלא נכונה.

הארכיטקטורה — Agentic Retrieval

לא צנרת RAG קלאסית. במקום לשלוח top-k קטעים בכל שאלה ולקוות שפגעתם, אתם נותנים ל-Claude כלים על אינדקס מובנה, והוא מחליט מה לקרוא.

זה מה שאתם מממשים:

```
def list_sections() -> list[dict]:
    """Return the table of contents: id, title, source document, page."""

def get_section(section_id: str) -> str:
    """Return the full text of one section."""

def search(query: str, limit: int = 5) -> list[dict]:
    """Lexical search over SQLite FTS5 (BM25). Returns section ids + snippets."""

def get_course_table(year: int, semester: int) -> list[dict]:
    """Return one course table as structured rows."""
```

כלי	מה הוא מחזיר
<code>list_sections</code>	תוכן העניינים — מזהה, כותרת, מסמך מקור, עמוד
<code>get_section</code>	הטקסט המלא של סעיף אחד
<code>search</code>	חיפוש לקסיקלי, BM25 מעל FTS5 — מזהי סעיפים וקטעי הקשר
<code>get_course_table</code>	טבלת קורסים אחת, כשורות מובנות

למה ככה ולא top-k רגיל:

- שאלות צבירה (כמה נ"ז סה"כ בשנה ג' ?) דורשות טבלה שלמה, לא שלושה קטעים דומים סמנטית. `get_course_table` מחזיר את כולה.
 - המסמכים כבר מובנים — יש להם סעיפים ממוספרים וטבלאות עם כותרות. לזרוק את המבנה הזה ולחתוך לחלונות של 1500 תווים זה להשמיד מידע שקיבלתם בחינם.
 - עברית ו-embeddings זה קרב לא נעים. ככה אתם לא נכנסים אליו.
- החיפוש הלקסיקלי עדיין שלכם לממש — BM25 אמיתי מעל FTS5, פשוט עטוף ככלי שהמודל קורא לו כשהוא מחליט, במקום צנרת קבועה שרצה תמיד.

דרישות חובה

ארבע דרישות. פרויקט שלא עומד באחת מהן לא מוגש.

1. ציטוט מקור בכל תשובה

כל תשובה מציינת מאיזה מסמך ומאיזה סעיף/עמוד היא נלקחה. תשובה בלי מקור נחשבת תשובה שגויה, גם אם היא נכונה.

2. סירוב כשאין תשובה

כששאלה לא נענית מהמסמכים, התשובה היא לא מופיע במסמכים — ולא ניחוש, ולא ידע כללי על מכללות.

3. Prompt Caching

חובה. בלולאת כלים כל פנייה שולחת מחדש את כל מה שקדם לה, ובלי cache אתם משלמים על אותו טקסט שוב ושוב. הצורה הנכונה: `cache_control` על הבלוק האחרון של כל תור. הגדרות הכלים ו-system prompt נשארים תחילית יציבה שנשמרת במטמון בין שאלות.

4. תקרת איטרציות

`max_iterations` בין 5 ל-8 על לולאת הכלים. סט כלים מעוצב גרוע שולח את המודל חמש-עשרה סיבובים לפני שהוא מוותר — זה מה ששורף תקציב. בנוסף, תדווחו ב-README כמה קריאות כלים במומצע המערכת שלכם עושה לשאלה. זה מדד הנדסי אמיתי, קל למדוד, וצוות שהממוצע שלו 9 יודע שיש לו בעיה בעיצוב הכלים.

שלבי העבודה

הפרויקט מתפצל לארבעה מסלולים שרצים במקביל. שלב 1 הוא החוזה בין כולם — ברגע שיש טקסט מחולץ נקי, שאר המסלולים יכולים לעבוד עצמאית.

יום 0 — ביחד, שעה

תקראו את שני ה-PDFים כבני אדם. תחליטו מה סטודנט אמיתי באמת שואל.

שלב 1 — חילוץ ומבנה

חילוץ הטקסט, תיקון כיווניות, ניקוי תווי בקרה, זיהוי סעיפים וטבלאות, בניית אינדקס ה-FTS5.

שער מעבר: תפתחו את הפלט ותקראו אותו לפני שממשיכים.

שלב 2 — שכבת המודל

הגדרת הכלים, לולאת ה-tool use, ה-system prompt, ה-caching, תקרת האיטרציות, התנהגות הסירוב.

שלב 3 — ממשק

Streamlit, עברית ו-RTL, שאלה נכנסת ← תשובה מבוססת עם ציטוט יוצאת.

שלב 4 — הערכה

הרצת עשר שאלות ההערכה, תיעוד הפלטים בפועל, ניתוח הכשלים.

סט שאלות ההערכה

עשר שאלות, זהות לכל הצוותים. ההגשה כוללת טבלה עם הפלט בפועל של המערכת שלכם לכל שאלה.

#	שאלה	סוג
1	מהו מספר הקורס של "מבוא למדעי המחשב" וכמה נ"ז הוא מזכה?	עובדתי
2	מה משך הלימודים המקובל לתואר ראשון לפי התקנון?	עובדתי
3	אילו קורסים נלמדים בשנה ב' סמסטר 4, וכמה נ"ז כל אחד?	טבלה
4	מהן דרישות הקדם של הקורס "אלגוריתמים 1"?	טבלה
5	אילו קורסי חובה יש במסלול "עיבוד אותות ולמידה חישובית"?	טבלה
6	כמה נ"ז סה"כ נדרשות בשנה ג'?	צבירה
7	כמה קורסים בשנה א' יש להם דרישות קדם?	צבירה
8	מה ההבדל בין מסלול חד-חוגי לדו-חוגי, ואיזה מהם רלוונטי לתואר במדעי המחשב?	חוצה מסמכים
9	לפי התקנון, מה התנאים לפריסת לימודים לארבע שנים?	חוצה מסמכים
10	מהן שעות הקבלה של המזכירות האקדמית?	לא במסמכים

שאלה 10 היא המבחן האמיתי. התשובה הנכונה היחידה היא שהמידע לא מופיע במסמכים.

כישלון גלוי שווה יותר מהצלחה שקטה. צוות שמדווח 6 מתוך 10 ומסביר בדיוק למה כל אחת נכשלה מקבל ציון גבוה יותר מצוות שמדווח 8 בלי ניתוח.

מה מגישים

ריפו ציבורי ב-GitHub ובו:

- קוד** — חילוץ, אינדוקס, כלים, לולאה, ממשק
 - README.md** הכולל: - הוראות הרצה שעובדות ממכונה נקייה - פסקה שמנמקת את עיצוב הכלים שלכם — למה הסעיפים מפוצלים ככה, מה `search` מחזיר ולמה - טבלת ההערכה עם הפלטים בפועל וניתוח הכשלים - ממוצע קריאות הכלים לשאלה - השוואה קצרה: הריצו 3–4 שאלות גם דרך גישה שמכניסה את כל המסמכים ל-context בבת אחת, ותדווחו איפה כל גישה מנצחת
 - הדגמה** — סרטון של 3 דקות, או הדגמה חיה בכיתה
- ההשוואה בסעיף 2 עולה לכם שעה והיא המשפט הכי בעל ערך בכל ההגשה. הקורפוס הזה נכנס כולו לחלון הקשר אחד — כדאי שתדעו מה קניתם בזה שבניתם שליפה.

מפתחות API ותקציב

מפתח ה-API של הפרויקט מגיע ממני, עם תקרת הוצאה לכל צוות.

1. **המפתח הזה הוא לאפליקציה שלכם — לא ל-Claude Code.** אל תגדירו אותו כ-`ANTHROPIC_API_KEY` גלובלי במערכת. Claude Code שורף סדר גודל יותר טוקנים מהצ'אטבוט שתבנו, והוא ירוקן את התקרה תוך שעות. ל-Claude Code תשתמשו במנוי שלכם.
2. **לא מעלים לאוויר.** אפליקציית Streamlit ציבורית עם המפתח שלי ובלי אימות היא דרך לאבד כסף אמיתי בלילה אחד. הפרויקט רץ מקומית: `streamlit run`, המפתח ב-`.env`, ו-`.env` ב-`.gitignore`.
3. **תעקבו אחרי מה שאתם שורפים.** התקרה מגינה עליי, לא עליכם — צוות שמגיע אליה ביום השני נשאר בלי מפתח. מעבר לזה הפרויקט לא עולה כסף. כל הסטאק חינומי, אין תשתית לשלם עליה, ואין ספק שני.

הערה על עברית

עברית בפרויקט הזה היא לא דקורציה — היא מקור אמיתי של קושי הנדסי, בשלושה מקומות שונים לפחות. אתם תיתקלו בהם. זו לא תקלה בפרויקט, זה הפרויקט. כשאתם נתקעים על משהו שנראה כמו ג'יבריש — תעצרו, תסתכלו על הבייטים, ותבינו מה קרה לפני שאתם כותבים תיקון. [:-1]

על מחרוזת מעורבת עברית-אנגלית-מספרים זה לא תיקון, זה באג חדש.

נתקעתם?

תביאו את השגיאה למפגש. חצי מהקורס הוא לראות איך מפרקים תקלה אמיתית — ותקלות בפרויקט הזה יהיו אמיתיות.